

2장 리마인드 + 3.1 정밀 분석

현재 AI와의 연결, 그리고 최신 AI 트렌드

3.1은 개념 정의가 대부분이라 "책 내용 분석"만으로는 얻을 게 적어요. 그래서 각 개념을 ① 현재 AI에서 어떻게 쓰이는가(참고 기술·논문) ② 왜 알아야 하는가 ③ 알면 무엇을 할 수 있고 어떻게 응용하는가의 세 관점으로 풀었어요. 마지막 Part 3는 별도의 최신 AI 트렌드 정리에요. (2026년 10월 8일 기준)

구성	내용
Part 1	2장 퍼셉트론 리마인드 (핵심, 진리표와 가중치, XOR, 스터디 질문으로 정리한 "학습"의 의미)
Part 2	3.1 정밀 분석: 개념 7개 × 세 관점 (1·2·3장 범위)
Part 3	최신 AI 트렌드 7가지와 1-3장과의 연결

Part 1 · 2장 퍼셉트론 리마인드

$$y = 1 \text{ (} b + w_1x_1 + w_2x_2 > 0 \text{)}, \quad y = 0 \text{ (그 외)} \quad [\text{식 2.2}]$$

절	핵심	한 줄 의미
2.1	w는 입력의 중요도, b(= -θ)는 얼마나 쉽게 1이 되는지	매개변수 두 종류의 역할 구분
2.2-2.3	AND·NAND·OR는 같은 식, 다른 (w ₁ , w ₂ , b)	구조는 같고 값만 바꾸면 다른 일을 함 → 파인튜닝의 원리
2.4	단층은 직선 하나로만 나눔 → XOR 불가 (선형/비선형)	모델의 표현 한계
2.5	NAND·OR의 출력을 AND의 입력으로 → XOR 가능	앞층 출력 = 다음 층 입력, 다층 퍼셉트론
2.6	2층(+비선형 활성화)이면 임의의 함수를 표현 가능	표현할 수 있다 ≠ 학습으로 찾을 수 있다

게이트	(w ₁ , w ₂ , b) 예시	진리표
AND	(0.5, 0.5, -0.7)	1,1일 때만 1
NAND	(-0.5, -0.5, 0.7)	AND의 반대
OR	(0.5, 0.5, -0.2)	하나라도 1이면 1
XOR	단층으로는 없음	서로 다를 때만 1

```
def XOR(x1, x2):
    s1 = NAND(x1, x2) # 1층
    s2 = OR(x1, x2)    # 1층
    return AND(s1, s2) # 2층
```

스터디 질문으로 정리한 "학습"의 의미

스터디 질문 · 2장에서 가중치는 사람이 직접 정했나?

네. 진리표를 보고 조건(AND라면 $w_1 + w_2 > \theta$, $w_1 \leq \theta$, $w_2 \leq \theta$)을 만족하는 값을 사람이 찾았어요. 답은 무한히 많아요(p.49). p.50 NOTE: "학습이란 적절한 매개변수 값을 정하는 작업이며, 사람은 퍼셉트론의 구조(모델)를 고민하고 컴퓨터에 학습할 데이터를 주는 일을 한다." 참고로 1958년 원래 퍼셉트론에는 "틀린 만큼 고치는" 학습 규칙($w \leftarrow w + \eta(t - y)x$)이 있었고, 직접 돌려 보니 AND 진리표를 5번째 반복에서 $w = (0.1, 0.2)$, $b = -0.2$ 로 스스로 찾았어요. 하지만 XOR처럼 직선으로 못 나누는 문제는 이 규칙으로 못 풀어요.

다듬은 정의

학습이란, 사람이 정한 식의 틀(모델) 안에서 컴퓨터가 데이터를 보며 틀린 정도(손실)가 가장 작아지도록 매개변수(가중치·편향)를 반복해서 조정하는 과정이다. 기준값(편향)도 컴퓨터가 찾는다. 학습이 끝나면 정보 자체가 아니라 **패턴이 압축된 매개변수**가 남고, 이를 파일로 저장·배포하면 다른 사람도 새 입력에 쓸 수 있다.

사람이 정함	컴퓨터가 찾음	AI 활용에서의 의미
식의 틀(모델), 데이터와 정답, 손실 함수	가중치 w , 편향 b	학습 후 w , b 는 고정 → 새 정보는 입력(프롬프트·RAG)으로 줌

2장 → 현재 AI

트랜스포머 블록의 FFN(MLP)이 2층 퍼셉트론 구조이고 블록 파라미터의 약 2/3를 차지해요(어텐션 $4d^2$, FFN $8d^2$). MoE는 이 MLP를 여러 개 두고 일부만 골라 써요(Mixtral). 층을 깊고 넓게 쌓고 데이터를 함께 키우면 성능이 예측 가능하게 오른다는 것이 스케일링 법칙이에요(Kaplan 2020, Chinchilla 2022).

Part 2 · 3.1 정밀 분석 (1·2·3장 범위)

3.1(p.63-68)에서 뽑은 개념 7개를 차례로 봐요. 각 개념마다 책 내용과 1·2장 연결을 먼저 짧게 정리하고, 이어서 세 관점으로 분석해요.

개념 1. 가중치를 데이터로 학습하는 신경망 3장 도입 p.63

책: 퍼셉트론의 나쁜 소식은 "가중치를 사람이 수동으로 정한다"는 것. 신경망은 가중치 매개변수의 적절한 값을 데이터로부터 자동으로 학습하는 능력이 있다. 3장은 식별(추론)만 다루고 학습은 4장부터. **1·2장 연결:** 2장 p.50 NOTE의 "학습 = 매개변수 정하기"가 3장에서 신경망의 정체성이 돼요.

① 현재 AI에서의 쓰임 · 참고

- 모든 LLM은 수십억~수조 개 가중치를 데이터로 학습해요. 학습(훈련)과 추론(사용)은 비용과 시점이 완전히 다른 두 단계예요.
- 참고: [Rumelhart-Hinton-Williams 1986, 오차역전파](#) (5장의 원전), [Chinchilla](#)(데이터와 파라미터의 균형).

② 왜 알아야 하나

- "모델이 대화로 새로 배우지 않는 이유", "학습 시점 이후 정보를 모르는 이유"가 여기서 나와요.
- AI의 행동을 예측하는 가장 기본적인 사실이에요.

③ 알면 할 수 있는 것 · 응용

- 정보를 줄 방법을 고를 수 있어요: 값을 바꾸는 **파인튜닝(w)** vs 입력을 바꾸는 **프롬프트·검색 증강(x)**. 대부분은 x로 해결하고 파인튜닝은 마지막 수단이에요([Anthropic Contextual Retrieval](#)).
- AI 에이전트를 만들 때 "기억"은 가중치가 아니라 입력에 다시 넣는 정보라는 점을 설계에 반영할 수 있어요([Context engineering](#)).

개념 2. 입력층 · 은닉층 · 출력층 3.1.1 p.64

책: 은닉층의 뉴런은 입력층·출력층과 달리 사람 눈에 보이지 않는다. 뉴런이 연결되는 방식은 퍼셉트론과 달라진 것이 없다. **1·2장 연결:** 2장 XOR의 s_1, s_2 가 은닉층이에요. 1장 넘파이 배열로 보면 은닉층은 중간 계산 결과 배열(z_1, z_2)이에요.

① 현재 AI에서의 쓰임 · 참고

- LLM 내부를 오가는 벡터를 **hidden states**, 그 크기를 **hidden_size**라고 불러요(BERT-base 768, [HF 문서](#)).
- "보이지 않는" 은닉층을 들여다보는 **해석 가능성** 연구: 뉴런 하나가 여러 의미를 섞어 갖는 중첩([Toy Models of Superposition](#)), 은닉 표현을 사람이 읽을 수 있는 특징으로 분해([Scaling Monosemanticity](#)), 계산 경로 추적([On the Biology of a LLM](#)), 2026년 진행 상황([Circuits Updates](#)).

② 왜 알아야 하나

- 모델이 "이해한 내용"은 은닉층의 숫자 배열(표현)로만 존재해요. 임베딩, 벡터 검색, 해석 가능성이 모두 이 은닉 표현을 다루는 기술이에요.
- 모델 설정 파일(`config.json`)의 용어를 읽을 수 있어요.

③ 알면 할 수 있는 것 · 응용

- **임베딩 활용**: 문장을 모델에 넣고 은닉층 벡터를 꺼내 비슷한 문서 찾기, 분류, 추천에 써요. RAG의 "벡터 검색"이 이것이에요.
- **전이 학습**: 학습된 모델의 은닉층을 재사용하고 출력층만 새로 학습해 내 문제(예: 내 데이터 분류)에 맞출 수 있어요.
- **모델 크기 계산**: hidden_size와 층 수만 알면 대략적인 파라미터 수와 메모리를 계산할 수 있어요(GPT-2 small: d = 768, 12블록 → 약 124M).

개념 3. 층을 세는 기준 3.1.1 WARNING p.64

책: 그림 3-1은 3층으로 구성되지만 가중치를 갖는 층은 2개라 "2층 신경망". 문헌에 따라 "3층"이라 하기도 하니 주의. 층 번호는 파이썬 인덱스처럼 0부터. **1·2장 연결**: 2장 p.60 XOR의 "2층 퍼셉트론"과 같은 기준. 1장의 "인덱스는 0부터" (1.3.4)가 층 번호 규칙의 이유예요.

① 현재 AI에서의 쓰임 · 참고

- LLM 사양의 "N층(num_hidden_layers)"은 트랜스포머 블록 수예요. 블록 하나 안에 어텐션과 MLP가 다시 들어 있어요(Attention Is All You Need).

② 왜 알아야 하나

- 같은 모델을 자료마다 다르게 부르는 혼란을 피할 수 있어요. 코드(w_1 , w_2)와 그림의 층 번호를 대응시키는 기준이에요.

③ 알면 할 수 있는 것 · 응용

- AI에게 모델 구조를 요청할 때 "은닉층 2개, 각 64개 뉴런"처럼 **기준을 분명히** 말하면 원하는 구조가 정확히 나와요.
- 논문·모델 카드를 읽을 때 "layer"가 블록인지 선형층인지 구분해서 비교할 수 있어요.

개념 4. 편향 = "항상 1인 입력"의 가중치 3.1.2 그림 3-3 p.65-66

책: b는 뉴런이 얼마나 쉽게 활성화되는지, w는 각 신호의 영향력을 제어. 그림 3-3은 입력이 1인 뉴런(회색)과 가중치 b로 편향을 명시. **1·2장 연결**: 2장 2.3.2에서 θ 를 $-b$ 로 옮긴 것을 그림으로 표현. 1장 브로드캐스트가 행렬 계산에서 B를 데이터 수만큼 늘려 줘요(3.4의 $A = XW + B$).

① 현재 AI에서의 쓰임 · 참고

- PyTorch `nn.Linear(in, out, bias=True)` 가 "가중합 + 편향"(문서).
- 편향은 빼도 구조가 무너지지 않아서, Llama 계열은 선형층의 편향을 기본으로 쓰지 않아요(attention_bias, mlp_bias 기본 False, HF Llama 문서).

② 왜 알아야 하나

- 편향도 학습되는 매개변수라는 것, 그리고 편향이 결정 경계(직선)를 원점에서 떨어뜨려 옮기는 역할이라는 것을 이해할 수 있어요.

③ 알면 할 수 있는 것 · 응용

- 파라미터 수를 정확히 셀 수 있어요: 입력 n, 출력 m인 층은 $n \times m + m$ 개(3.4.3 신경망은 총 23개).
- 모델 설정의 bias 옵션이 무엇을 바꾸는지 이해하고 코드를 읽을 수 있어요.

개념 5. 조건 분기를 함수 $h(x)$ 로 분리 3.1.2 식 3.2-3.3 p.66

책: 식 3.1의 "0을 넘으면 1, 아니면 0"을 $h(x)$ 라는 함수 하나로 묶어 $y = h(b + w_1x_1 + w_2x_2)$ 로 표현. **1·2장 연결**: 1장의 if 문(1.3.7)을 함수(1.3.9)로 묶은 것. 3.2.2에서는 불리언 배열(1장 1.5.6)로 if 문 없이 구현. 스캔 메모의 "indicator function"(지시 함수)이 바로 이 h 예요.

① 현재 AI에서의 쓰임 · 참고

- 함수로 분리했기 때문에 h 를 바꾸는 연구가 가능해졌어요: ReLU([Glorot 2011](#)), 자동 탐색으로 찾은 Swish([Searching for Activation Functions](#)), LLM의 GELU·SwiGLU([GLU Variants](#)).
- 더 나아가 활성화 함수 자체를 학습하는 구조도 나왔어요: KAN ([Kolmogorov-Arnold Networks, 2024](#))은 고정된 h 대신 연결마다 학습 가능한 함수를 뒀요.

② 왜 알아야 하나

- "모델 = 바꿔 끼울 수 있는 부품의 조합"이라는 설계 감각의 출발점이에요.
- if 문을 배열 연산으로 바꾸는 벡터화는 딥러닝 코드 속도의 핵심이에요.

③ 알면 할 수 있는 것 · 응용

- AI가 짠 코드에서 `nn.ReLU()` 를 `nn.GELU()` 로 바꾸는 등 한 줄 수정으로 실험할 수 있어요.
- 바이브 코딩에서 for-if로 원소를 하나씩 처리하는 코드를 보면 "넘파이/PyTorch 벡터 연산으로 바꿔줘"라고 요청해 속도를 크게 높일 수 있어요(직접 측정: 신경망 한 층 기준 약 500배).

개념 6. 뉴런 = 가중합 → 활성화의 2단계 (합성함수) 3.1.3 식 3.4~3.5, 그림 3-4 p.66~67

책: 활성화 함수는 입력 신호의 총합을 출력 신호로 변환하는 함수. $a = b + w_1x_1 + w_2x_2$, $y = h(a)$. 그림 3-4는 뉴런 안의 $a \rightarrow h() \rightarrow y$ 를 명시. 뉴런 = 노드. **1·2장 연결:** 2장의 "앞층 출력 → 다음 층 입력"과 합치면 신경망 전체가 $y = \sigma(h(h(xW_1+b_1)W_2+b_2)W_3+b_3)$ 라는 **합성함수**가 돼요(스터디 질문에서 확인).

① 현재 AI에서의 쓰임 · 참고

- 모든 신경망 코드의 기본 패턴 "Linear → 활성화". 트랜스포머 FFN은 "Linear → GELU/SwiGLU → Linear".
- 합성함수이기 때문에 학습은 **연쇄법칙**으로 기울기를 계산해요(5장). PyTorch의 자동 미분 `loss.backward()` 가 이를 자동화([Autograd 튜토리얼](#)).
- FFN 층이 지식을 저장하는 키-값 메모리처럼 작동한다는 연구([Geva 2021](#)).

② 왜 알아야 하나

- a (선형, 매개변수 있음)와 h (비선형, 매개변수 없음)의 역할 분담을 알면 "어디가 학습되고 어디가 고정인지"가 보여요.
- 비선형 h 가 없으면 층을 쌓아도 하나와 같다는 것(3.2.6)을 이해하는 기반이에요.

③ 알면 할 수 있는 것 · 응용

- 모델 코드의 `forward()` 를 "가중합과 활성화가 몇 번, 어떤 순서로 합성되나"로 읽을 수 있어요.
- 디버깅: 각 단계의 a 와 z 를 출력해 값이 폭발(∞)하거나 모두 0이 되는 지점을 찾을 수 있어요.
- 좌표 평면 관점: 각 층은 "다음 층이 직선으로 나눌 수 있게 평면을 다시 그리는 함수"(스터디 시각화 페이지 7번).

개념 7. 단순 퍼셉트론 vs 다층 퍼셉트론(MLP) 3.1.3 WARNING p.68

책: 단순 퍼셉트론 = 단층 + 계단 함수. 다층 퍼셉트론 = 여러 층 + 시그모이드 등 매끈한 활성화 함수(= 신경망). "활성화 함수가 퍼셉트론에서 신경망으로 가기 위한 길잡이". **1·2장 연결:** 2장의 XOR 회로는 구조는 다층이지만 계단 함수라 학습이 어려워요. 매끈함이 추가되어야 신경망이 돼요.

① 현재 AI에서의 쓰임 · 참고

- "MLP"는 지금도 핵심 부품 이름이에요. 트랜스포머의 MLP 블록, MoE의 각 전문가(expert)가 모두 MLP예요 (Mixtral).
- 2층 + 비선형이면 임의의 함수를 근사할 수 있다는 이론적 근거(보편 근사 정리).

② 왜 알아야 하나

- "퍼셉트론"이라는 같은 단어가 1958년 장치와 현대 MLP를 모두 가리켜서, 맥락을 구분해야 자료를 정확히 읽을 수 있어요.

③ 알면 할 수 있는 것 · 응용

- 표 형태 데이터(엑셀, CSV)의 분류·예측에는 지금도 작은 MLP가 충분한 경우가 많아요. 바이브 코딩으로 "입력 n개, 은닉층 2개인 MLP로 분류 모델을 PyTorch로 만들어줘"처럼 요청해 직접 모델을 만들어 볼 수 있어요.
- MoE 모델 사양("30B 중 3B 활성화")을 MLP 전문가 몇 개를 골라 쓰는지로 해석할 수 있어요.

3.1 종합: 개념 → 역할 → 지금 쓰이는 곳

3.1의 개념	신경망에서의 역할	지금 쓰이는 곳
자동 학습	w, b를 데이터로 찾음	모든 모델 학습, 파인튜닝 vs 프롬프트 판단
은닉층	입력을 쓸모 있는 표현으로 바꿈	임베딩·벡터 검색, 해석 가능성 연구
층 세는 기준	구조를 정확히 소통	모델 사양, 논문 비교
편향	경계를 옮기는 매개변수	<code>nn.Linear</code> , Llama의 <code>bias=False</code>
함수 h로 분리	부품 교체 가능	ReLU·GELU·SwiGLU, KAN
가중합 → 활성화	뉴런 계산 = 합성함수	FFN, 연쇄법칙과 자동 미분
단순/다층 퍼셉트론	매끈함 = 학습 가능	MLP 블록, MoE 전문가

Part 3 · 최신 AI 트렌드 (2026년 10월 기준)

공식 발표와 주요 보고서를 중심으로 정리했어요. 블로그·뉴스레터 출처는 그렇게 표시했어요. 각 트렌드마다 1-3장과의 연결과 우리가 할 수 있는 것을 붙였어요.

트렌드 1. 최상위 모델의 실력 차이가 좁아졌어요

- 2026년 3월 기준 Anthropic, xAI, Google, OpenAI, Alibaba, DeepSeek 6개 회사의 모델이 Arena 리더보드에서 **25 Elo 이내**에 몰려 있고, 미국·중국 최고 모델 차이는 2.7%예요(Stanford AI Index 2026).
- 2025년 최고 공개 가중치 모델과 최고 비공개 모델의 차이는 3.3%로, 2024년에 거의 좁혀졌다가 다시 벌어졌어요(같은 보고서).
- Humanity's Last Exam 점수가 1년 사이 30%p 올랐지만, 아날로그 시계 읽기는 최고 모델 50.1% vs 사람 90.1%처럼 능력이 고르지 않아요.

1-3장 연결과 활용

모델 구조(트랜스포머 = 어텐션 + MLP)가 비슷해지면서 차이는 데이터와 학습 방식에서 나요. "어느 회사 모델이 최고인가"보다 **내 작업에 맞는 크기·가격·속도로** 고르는 것이 중요해졌어요.

트렌드 2. AI 에이전트와 컴퓨터 사용이 급성장했어요

- 컴퓨터 사용 벤치마크 OSWorld에서 에이전트 정확도가 약 12% → **66.3%**로 올랐어요(AI Index 2026). 기업의 88%가 AI를 쓰지만 에이전트는 아직 초기 단계예요.
- 2026년 9월 28일 공개된 Claude Sonnet 5.5는 Terminal-Bench 4.0 70.6%(Sonnet 5는 10.3%), OSWorld 2.1 80.1%를 기록했어요([Anthropic](#)).
- 서버에이전트·위임 기능이 모델과 도구에 들어가고, 조직마다 자기 업무 흐름에 맞춘 에이전트를 만드는 흐름이에요([O'Reilly Radar 2026.9](#)).

1-3장 연결과 활용

에이전트는 고정된 가중치의 모델에 도구(파이썬 함수)·메모리·반복 루프를 붙인 프로그램이에요. 1장의 함수·딕셔너리(JSON)가 도구 호출의 형식이고, "가중치는 고정, 입력으로 행동을 바꾼다"(개념 1)가 에이전트 설계의 출발점이에요.

트렌드 3. 생각하는 양을 조절하는 "effort" 설정과 작업당 비용

- Claude Sonnet 5.5는 Low~Max의 effort 단계를 제공하고, 높을수록 더 오래 추론하고 더 꼼꼼히 검토하지만 작업당 비용이 커져요([Anthropic](#)).
- 같은 모델도 effort와 출력 길이에 따라 작업당 비용이 약 7배까지 달라진다는 분석이 있어요([Local AI Zone 블로그](#)). 요금표보다 "작업 하나를 끝내는 데 드는 비용"이 중요해지는 흐름이에요.

1-3장 연결과 활용

LLM은 토큰 하나를 만들 때마다 forward(3.4.3)를 한 번 실행해요. 더 오래 생각한다 = forward를 더 많이 돌린다 = 계산과 비용이 늘어난다. 간단한 작업은 낮은 effort, 어려운 판단만 높은 effort로 나누는 것이 실용적인 활용법이에요.

트렌드 4. 작고 효율적인 모델, 그리고 MoE

- 노트북이나 GPU 한 장에서 도는 공개 가중치 모델이 대형 모델에 근접했다는 주장이 이어지고 있어요(O'Reilly Radar 2026.9).
- 예: NVIDIA Nemotron 3.5 Lightning(전체 30B 중 3B 활성 MoE), IBM Granite 4.2(3B·8B·30B 추론 모델), Kimi K3(2.8T 중 104B 활성)를 64GB 노트북에서 초당 약 0.5토큰으로 돌린 사례(같은 출처).
- Stanford AI Index도 경쟁의 축이 비용과 신뢰성으로 옮겨 가고 있다고 정리해요.

1-3장 연결과 활용

"30B 중 3B 활성"은 MLP 전문가 일부만 골라 계산한다는 뜻(개념 7). 내 컴퓨터에서 돌릴 수 있는지는 파라미터 수 $\times$ dtype(1장)으로 계산해요: $8B \times 2\text{바이트(float16)} \approx 16GB$, 4비트 양자화면 약 4GB.

트렌드 5. 에이전트 인프라의 표준화

- MCP(Model Context Protocol) 2026-07-28 사양이 **무상태(stateless)**로 바뀌었어요. 세션과 초기 핸드셰이크가 없 어지고 요청마다 필요한 정보를 담아서, 서버 확장이 쉬워졌어요(MCP 공식 블로그). TypeScript·Python·Go·C# SDK 가 지원해요.
- 에이전트를 격리된 일회용 컨테이너에서 실행하는 Docker Sandboxes, 여러 회사가 지지하는 Agent Plugins 표준 등이 나왔어요(O'Reilly Radar 2026.9).
- 에이전트가 행동하기 전에 저렴하게 확인하는 작은 "결정 모델"이 여러 회사에서 나오고 있어요(Local AI Zone 블로그).

1-3장 연결과 활용

MCP 도구는 파이썬 함수에 데코레이터를 붙여 만들어요(MCP Python SDK). 1장 함수·딕셔너리만 알아도 나만의 도구를 만들어 AI에 연결해 볼 수 있어요.

트렌드 6. 신뢰와 안전이 더 중요한 과제가 됐어요

- 기록된 AI 사고가 2024년 233건 → 2025년 362건으로 늘었고, 26개 주요 모델의 환각률은 테스트에 따라 22~94%로 차이가 컸어요. 기업 투명성 지수 평균은 58 → 40으로 떨어졌어요(AI Index 2026).
- 최상위 성능 모델 일부가 보안상 이유로 검증된 사용자에게만 제한 공개되는 흐름도 있어요(Local AI Zone 블로그).

1-3장 연결과 활용

출력층의 소프트맥스는 "그럴듯한 다음 단어"의 확률을 계산할 뿐 사실 여부를 보장하지 않아요(3.5). 그래서 중요한 사실은 출처를 요청하고 직접 확인하는 습관이 필요해요(Claude Docs: Reduce hallucinations).

트렌드 7. 사용 확산과 투자

- 생성형 AI는 3년 만에 세계 인구 기준 53% 보급, 미국 고등학생·대학생 5명 중 4명이 과제에 AI를 써요. 전 세계 기업 AI 투자는 두 배 이상으로 늘었고, AI 연산 능력은 2022년 이후 매년 3.3배씩 커졌어요(AI Index 2026).

1-3장 연결과 활용

AI를 쓰는 사람은 많아졌지만, 원리를 알고 결과를 검증하며 쓰는 사람은 여전히 적어요. 이 스터디에서 배우는 내용(가중치 고정, 확률 출력, shape, 비선형)이 그 차이를 만드는 기초예요.

참고 자료

교재: 「밑바닥부터 시작하는 딥러닝」 사이토 고키, 2장(p.47-62), 3장(p.63-105)

보고서·공식 발표: [Stanford HAI, 2026 AI Index Report](#) (요약: [UNU C3](#)) · [Introducing Claude Sonnet 5.5](#) · [MCP 2026-07-28 Specification](#) · [O'Reilly Radar Trends, September 2026](#) · [Local AI Zone, October 2026 Model Updates](#) (블로그)

논문·기술 문서: [Rumelhart et al. 1986](#) · [Glorot et al. 2011 \(ReLU\)](#) · [Swish](#) · [GELU](#) · [GLU Variants](#) · [KAN](#) · [Attention Is All You Need](#) · [FFN = Key-Value Memories](#) · [Mixtral of Experts](#) · [Scaling Laws](#) · [Chinchilla](#) · [Toy Models of Superposition](#) · [Scaling Monosemanticity](#) · [On the Biology of a Large Language Model](#) · [Circuits Updates June 2026](#) · [PyTorch nn.Linear](#) · [HF Llama](#) · [HF BERT](#) · [Contextual Retrieval](#) · [Effective context engineering](#)